

운영체제 (Operating Systems)

Term Project

공항 체크인 카운터 시뮬레이션

멀티레벨 CPU 스케줄링 시뮬레이터 설계 및 구현

1. 프로젝트 개요

특정 항공사가 5개의 체크인 카운터를 운영하며, 1등석(First), 비즈니스석(Business), 일반석(Economy) 승객이 혼재하는 환경에서 **전체 승객의 Average Turnaround Time(ATT)을 최소화**하는 것을 목표로 한다. 본 프로젝트에서 학생들은 운영체제의 CPU 스케줄링 개념을 항공 체크인 시스템에 매핑하여, 복수의 스케줄링 알고리즘을 조합한 **자신만의 비선점형(Non-preemptive) 멀티레벨 스케줄링 정책**을 제안하고 시뮬레이터를 구현한다.

1.1 주요 전제 조건

- **비선점형(Non-preemptive) 처리:** 카운터에서 서비스를 시작한 승객은 완료될 때까지 중단되지 않는다.
- **Context Switching Overhead 없음:** 승객 교체 시 전환 비용은 0 이다.
- **동일 처리 속도:** 모든 카운터 직원의 처리 능력은 동일하다.
- **입력 데이터:** 승객 ID, 도착 시간, 승객 등급, 서비스 시간의 4 개 항목만 사용한다.
- **성능 지표:** Average Turnaround Time(ATT) 단일 지표만 사용한다.

1.2 CPU 스케줄링 ↔ 체크인 시스템 매핑

OS 개념	체크인 시스템 대응	비고
CPU	체크인 카운터 (5개)	멀티프로세서 환경
Process	승객 (Passenger)	각 승객 = 하나의 프로세스
Burst Time	서비스 시간 (Service Time)	입력으로 직접 주어짐
Priority	승객 등급	FIRST(1) > BUSINESS(2) > ECONOMY(3)
Ready Queue	대기열	등급별 분리 또는 통합 가능
Non-preemptive	서비스 시작 후 중단 없음	완료까지 카운터 점유

2. 시스템 환경

2.1 카운터 구성

총 5개의 카운터가 존재한다. 모든 카운터의 처리 속도는 동일하며, 승객 교체 시 전환 비용은 없다.

카운터 ID	전용 등급	비고
C1	First 전용	1등석 승객만 처리
C2	Business 전용	비즈니스석 승객만 처리
C3	Economy 전용	일반석 승객만 처리
C4	공용 (Flex)	모든 등급 처리 가능
C5	공용 (Flex)	모든 등급 처리 가능

전용 카운터 (C1, C2, C3): 해당 등급 승객만 처리 가능하다. 전용 카운터가 유ힴ(idle) 상태일 때 다른 등급 승객을 수용할 것인지 여부는 학생의 설계 대상이다.

공용 카운터 (C4, C5): 모든 등급의 승객을 처리할 수 있다. 어떤 등급의 승객을 우선 배정할 것 인지는 학생의 설계 대상이다.

2.2 입력 데이터 형식

각 승객은 다음 4개의 속성만을 가진다.

속성	타입	설명
passenger_id	String	고유 식별자 (예: 01)
arrival_time	Integer (0~60)	체크인 구역 도착 시간 (time unit)
class	Enum	FIRST(1) / BUSINESS(2) / ECONOMY(3)
service_time	Integer	체크인 소요 시간 (= CPU burst time)

3. 테스트 데이터셋

아래는 본 프로젝트에서 사용하는 기본 테스트 데이터셋이다. 총 50명의 승객으로 구성되며, 도착 시간은 0~58 범위이다. 구성 비율: First 8명(16%), Business 10명(20%), Economy 32명(64%).

passenger_id	arrival_time	class	service_time
P01	0	ECONOMY	7
P02	0	FIRST	12
P03	1	ECONOMY	5

P04	2	BUSINESS	9
P05	3	ECONOMY	4
P06	3	ECONOMY	8
P07	5	FIRST	15
P08	5	BUSINESS	10
P09	6	ECONOMY	3
P10	7	ECONOMY	6
P11	8	BUSINESS	11
P12	9	ECONOMY	5
P13	10	ECONOMY	9
P14	10	FIRST	14
P15	11	ECONOMY	4
P16	12	BUSINESS	8
P17	13	ECONOMY	6
P18	14	ECONOMY	7
P19	15	FIRST	13
P20	16	ECONOMY	3
P21	17	BUSINESS	10
P22	18	ECONOMY	5
P23	19	ECONOMY	8
P24	20	BUSINESS	7
P25	21	ECONOMY	4
P26	22	FIRST	11
P27	23	ECONOMY	6
P28	25	ECONOMY	9
P29	25	BUSINESS	8
P30	27	ECONOMY	5
P31	28	ECONOMY	7
P32	30	FIRST	12

P33	30	BUSINESS	9
P34	31	ECONOMY	3
P35	33	ECONOMY	6
P36	35	ECONOMY	8
P37	35	BUSINESS	10
P38	37	ECONOMY	4
P39	38	ECONOMY	7
P40	40	FIRST	14
P41	41	ECONOMY	5
P42	43	BUSINESS	9
P43	44	ECONOMY	6
P44	45	ECONOMY	3
P45	47	ECONOMY	8
P46	50	FIRST	11
P47	52	BUSINESS	7
P48	54	ECONOMY	5
P49	56	ECONOMY	9
P50	58	ECONOMY	4

3.1 데이터셋 통계 요약

항목	FIRST	BUSINESS	ECONOMY	전체
승객 수	8	10	32	50
평균 service_time	12.8	9.1	5.7	7.3
service_time 범위	11~15	7~11	3~9	3~15
총 service_time 합	102	91	181	374

4. 과제 요구사항

본 프로젝트는 3개 Phase로 구성된다.

Phase 1. 스케줄링 정책 설계서 (Design Document)

아래 항목을 포함하는 설계서를 작성하여 제출한다.

1. **큐 아키텍처 설계:** 몇 개의 큐를 사용할 것인가? 등급별 분리 큐, 통합 큐, 하이브리드 중 어떤 구조를 선택하며 그 이유는 무엇인가? 각 큐에 어떤 스케줄링 알고리즘을 적용하는가? 큐 간 관계(상위 큐 우선, 가중 배분 등)는 어떻게 정의하는가?
2. **알고리즘 조합 근거:** 최소 3 가지 이상의 비선점형 CPU 스케줄링 알고리즘을 조합하여 사용해야 한다. 각 알고리즘을 해당 위치에 배치한 이유를 OS 이론에 근거하여 서술한다.
3. **카운터 배정 전략:** 핵심 설계 결정: (a) 전용 카운터가 유휴 상태일 때 다른 등급 승객을 허용할 것인가? (b) Flex 카운터에 어떤 등급을 우선 배정할 것인가?

Phase 2. 시뮬레이터 구현 (Implementation)

4. **시뮬레이션 엔진:** 이산 사건 시뮬레이션 기반으로 구현한다. time unit 단위로 진행하며 모든 상태 변화를 로그로 기록한다.
5. **입력 처리:** Section 3 의 데이터셋을 파싱하여 시뮬레이션을 실행한다.
6. **스케줄러 모듈:** Phase 1 설계서의 알고리즘을 구현한다. 스케줄러는 교체 가능한 모듈(Strategy Pattern) 구조여야 한다. 이는 Phase 3 에서 Baseline 비교를 위해 필수이다.
7. **비선점 보장:** 어떤 상황에서도 처리 중인 승객이 중단되지 않아야 한다.
8. **사용 언어:** C, C++, Java, Python 중 택 1.
9. **출력:** 각 승객의 arrival_time, service_start_time, completion_time, turnaround_time 및 전체 ATT 를 출력한다.

Phase 3. 비교 분석 보고서

10. **Baseline 비교:** 자신의 스케줄러의 ATT 를 아래 3 가지 baseline 과 비교 분석한다 (직접 구현):
 - a) Baseline A: 단순 FCFS (모든 승객 단일 큐, 도착 순서대로 빈 카운터 배정)
 - b) Baseline B: 등급별 고정 우선순위 (First > Business > Economy, 등급 내 FCFS)
 - c) Baseline C: Non-preemptive SJF (등급 무관, service_time 짧은 승객 우선)

11. **ATT 비교 분석:** 기본 데이터셋에 대해 자신의 스케줄러와 3 가지 Baseline 의 ATT 를 비교하는 표와 그래프를 포함한다. 자신의 스케줄러가 baseline 대비 왜 더 좋은(또는 나쁜) ATT 를 보이는지 분석한다.
12. **Trade-off 분석:** ATT 를 줄이기 위해 어떤 요소가 희생되었는가? 등급별 ATT 차이는 어떠한가? 솔직한 한계 분석이 중요하다.

5. 성능 평가 지표

본 프로젝트의 유일한 성능 지표는 **Average Turnaround Time (ATT)**이다.

$$\text{Turnaround Time}(i) = \text{Completion Time}(i) - \text{Arrival Time}(i)$$

$$\text{ATT} = (1/n) \times \sum \text{Turnaround Time}(i), i = 1, 2, \dots, n \quad (n = 50)$$

시뮬레이터 출력 항목:

13. 각 승객별 arrival_time, service_start_time, completion_time, turnaround_time
14. 등급별(First / Business / Economy) 평균 Turnaround Time
15. 전체 **Average Turnaround Time (ATT)**

6. 팀 구성 및 제출

항목	내용
팀 인원	3인 1조 팀 단위 프로젝트(2인 팀도 허용, 1인 팀은 특별한 경우 허용)
역할 분담	설계/구현/분석 역할 명시, 모든 조원이 전체 코드 이해 필수
제출 방법	E-class에 지정된 기간내에 업로드
제출물	최종보고서(PDF 파일), 발표동영상

7. 참고 알고리즘 및 힌트

아래는 설계 시 참고할 수 있는 비선점형 스케줄링 알고리즘 후보이다. 이에 한정되지 않으며, 다른 알고리즘도 근거를 제시하면 사용 가능하다.

- **FCFS:** 단순하지만 Convoy Effect 발생 가능
- **Non-preemptive SJF:** 단일 CPU 이론적 최적 ATT, 긴 burst Starvation 위험

- **Non-preemptive Priority:** 등급 기반 우선순위, Aging 없이는 Starvation
- **Multi-Level Queue:** 등급별 분리 큐 + 큐 간 스케줄링 정책
- **MLFQ (비선점 변형):** 대기 시간에 따른 큐 간 Promotion
- **HRRN:** $\text{Response Ratio} = (\text{대기시간} + \text{burst}) / \text{burst}$. 비선점형 Aging 대표 기법
- **Lottery Scheduling:** 등급별 티켓 차등 배분, 확률적 공정성
- **Weighted Fair Queuing:** 등급별 가중치에 따른 카운터 시간 배분

8. 기타

- 과제 수행 시 새로운 알고리즘의 설계, 구현 시 생성형 AI 를 활용해서 문제를 해결하는 것을 적극 권장함
- 과제 수행 시 제약사항에 대한 요건을 융통성 있게 변경할 수 있으며, 최종 보고서에 그와 관련된 내용을 기술하면 됨
- 최종보고서와 발표동영상은 e-class 에 업로드(발표 동영상의 크기가 커서 e-class 에 업로드 불가능한 경우에는 이메일로 발표동영상만 따로 제출 가능)
- 최종보고서 출력본 1 부를 수업시간에 제출